

Setting up and Securing Cross-Tenant Access using Microsoft Entra ID

 minder-security.ghost.io/setting-up-and-securing-cross-tenant-access-using-microsoft-entra-id

Richard Minder

February 9, 2024



In this tutorial, I will showcase how to setup and tweak the Cross-Tenant Access Settings to ensure flawless collaboration between tenants.

Understanding Cross-Tenant Access#

Let's say, when we authenticate to our home tenant A, we have to complete an MFA challenge and the device compliance gets checked. As soon as all the criteria have been met, Entra saves these as successful in form of a cookie.

Tenant A



✓ MFA
✓ Device Compliance



Authentication

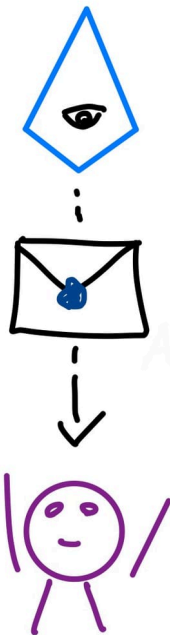


Customer Tenant

Tenant B



Tenant A

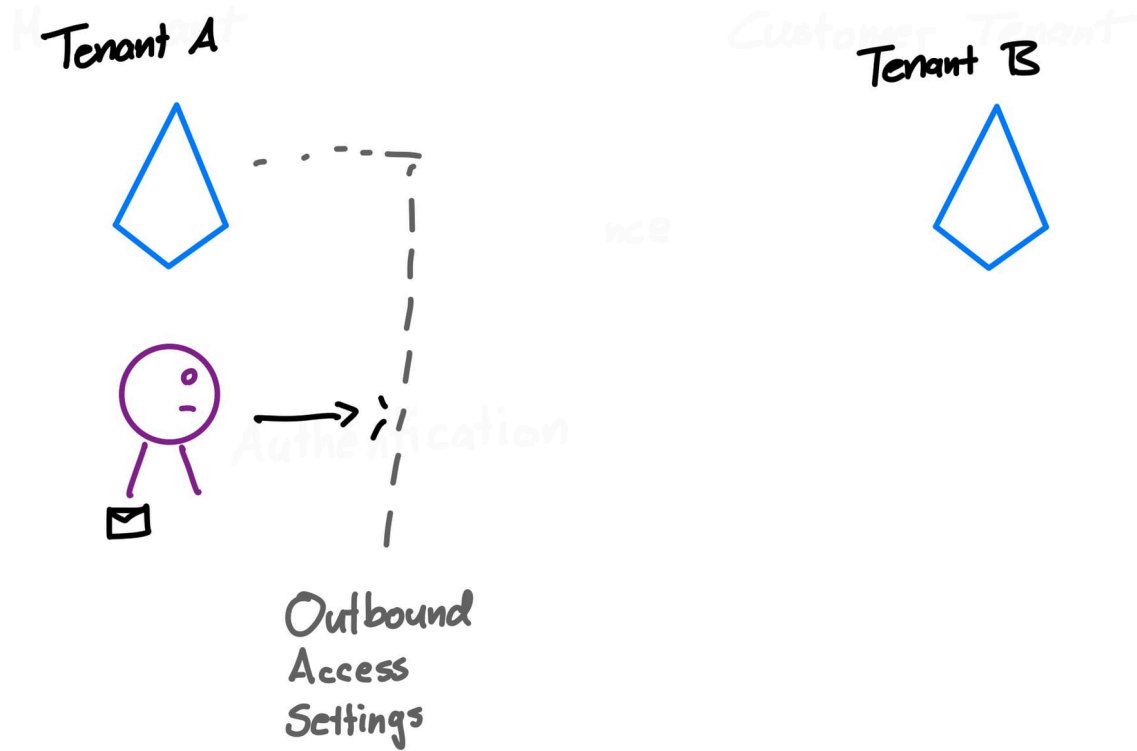


Customer Tenant

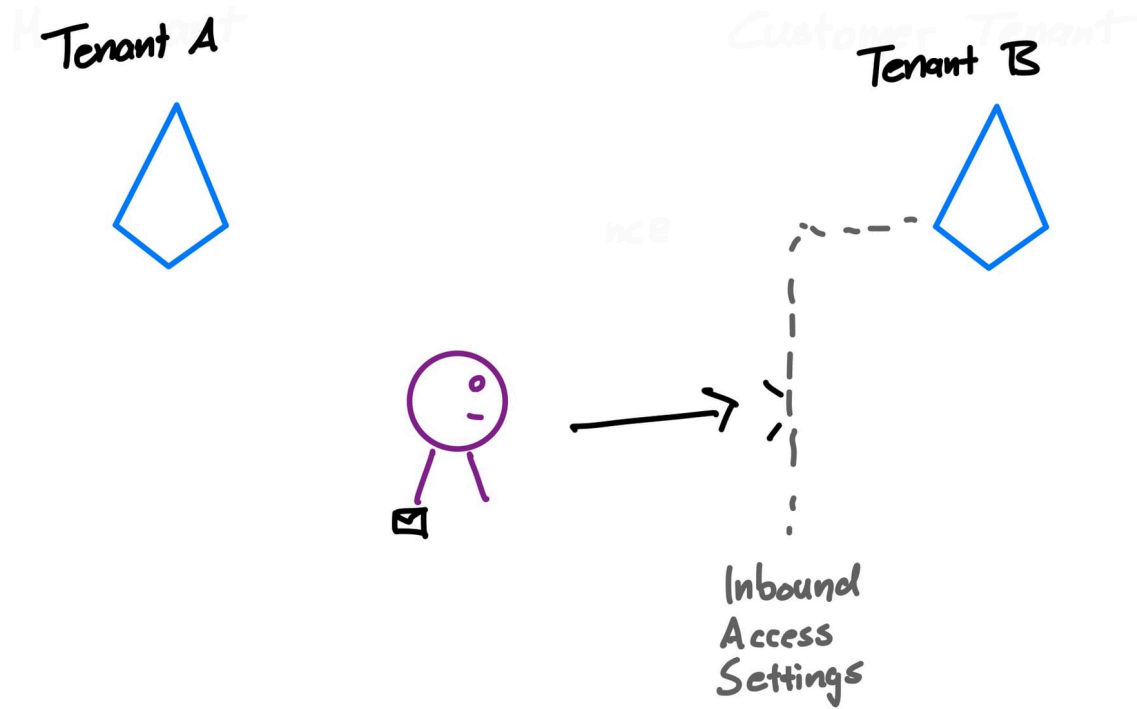
Tenant B



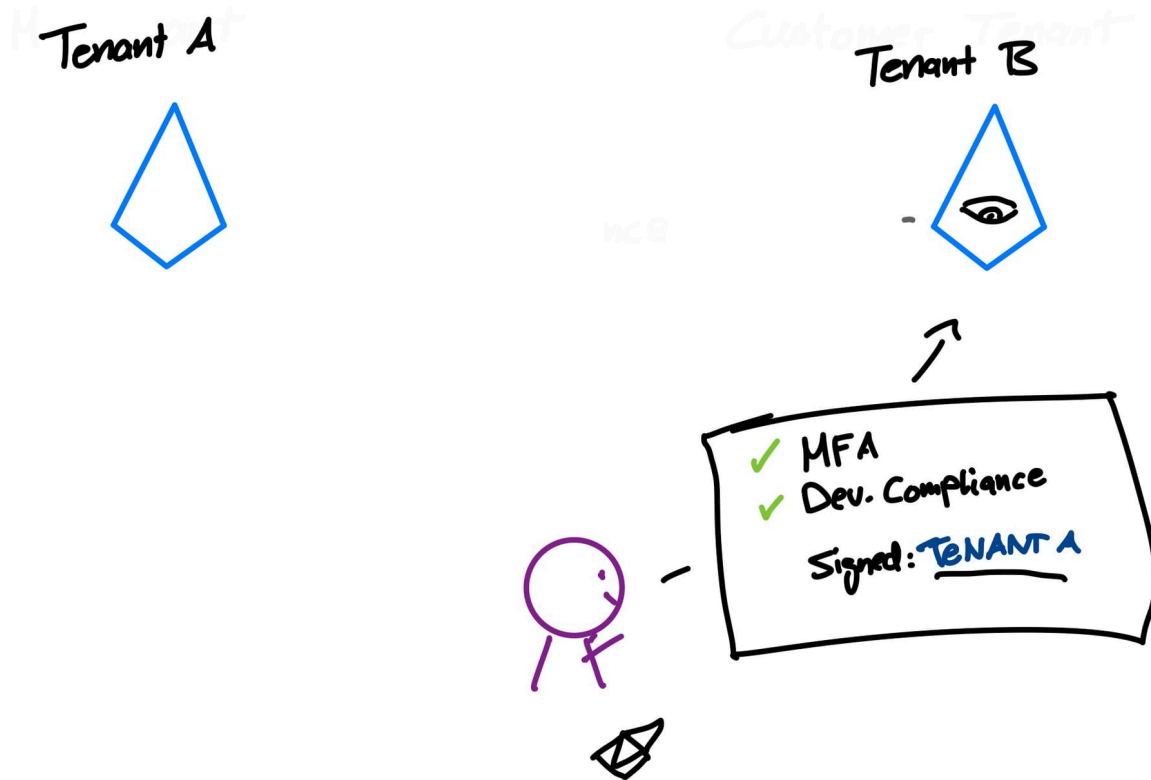
Now, we want to access resources in the target tenant B. Before we advance, our tenant A checks whether or not we are allowed to access that tenant.



After our tenant A allowed us to access tenant B, tenant B then also has settings either allowing or denying us access. If both tenant A's outbound access settings and tenant B's inbound access settings let us through, we proceed to the next step.



Now, since our tenant A already confirmed that we have a compliant device and completed the MFA challenge, we can forward these claims to tenant B. Tenant B has trust settings, which require external users to have these criteria checked.



After we passed all of these steps, we can proceed with our workflow and access tenant B's resources!



Now, let's look at how we can achieve a setup with security in mind.

Setting up Cross-Tenant Access securely#

In this tutorial, we will use the Entra Admin Center, but all the steps can also be followed using Entra ID on the Azure Portal.

Cross-tenant access refers to the ability for users from one Microsoft Entra organization (tenant) to interact with resources in another tenant. Whether it's sharing files, collaborating on projects, or accessing applications, cross-tenant access enables efficient collaboration between organizations.

Sign in to the **Microsoft Entra admin center** as a **Security administrator**.

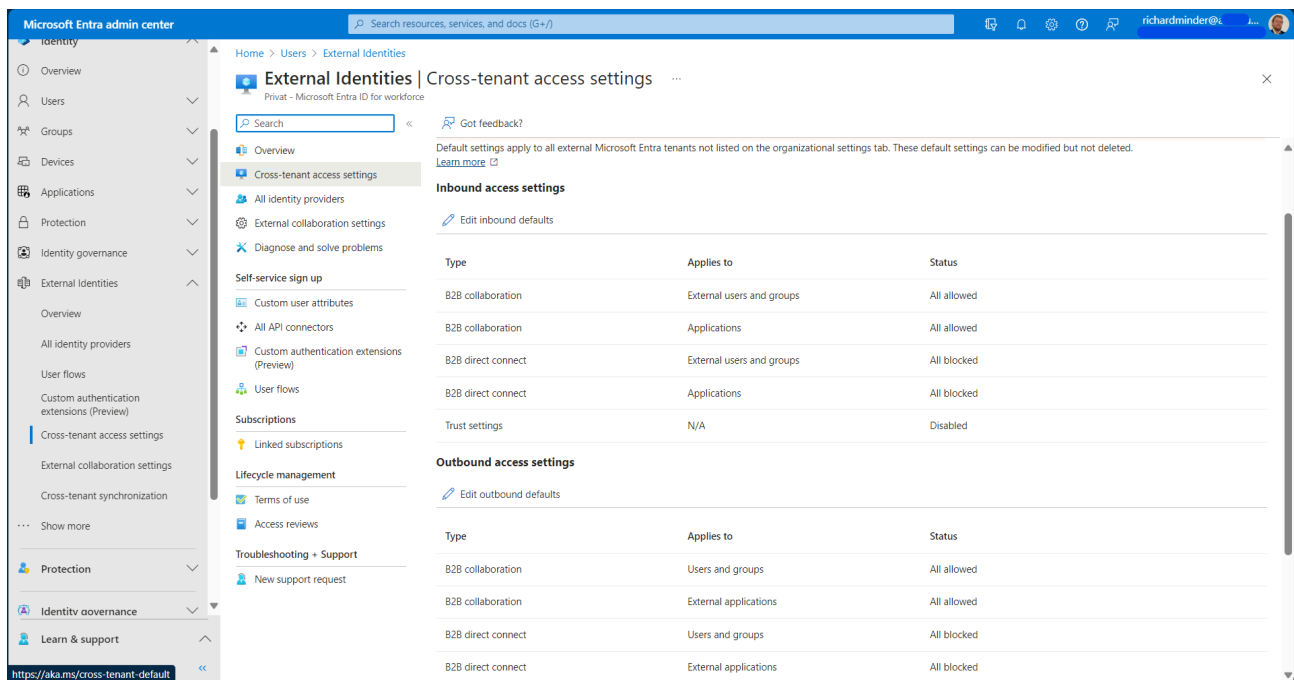
1 - Verify the Default Access Settings#

Before we add an organization and setup Cross-Tenant Access, we should confirm that the default access settings are set according to our security baselines. The **Default Access Settings** let us configure default security settings for cross-tenant access.

On the Entra Admin Center, navigate to:

Identity > External Identities > Cross-tenant access settings

Once there, it should look something like this:



On this page, you can configure the following settings to your liking:

- **Inbound Access Settings** let you configure:
 - Which users, groups or applications are granted access to your tenant
 - Redemption order for B2B Collaboration regarding identity providers
 - Whether you trust claims from the target tenant regarding MFA, Device Compliance and/or Entra Hybrid joined Devices
- **Outbound Access Settings** let you configure:
 - Which users, groups or applications from your tenant can access the target tenant
 - Whether you request MFA, Device Compliance and/or Entra Hybrid joined Devices when accessing the target tenant
- **Tenant Restrictions** let you configure:
 - Whether your users can access external applications from your network or devices using external accounts
 - Including accounts issued to them by external organizations and accounts they've created in unknown tenants.

Since these are default settings, they will automatically apply to all organizations without specific configuration profiles.

Be aware that these settings can influence existing collaboration

2 - Adding an Organization#

After we configured the default inbound and outbound access settings, we can add our first organization. This can be done by switching from the Default Settings Tab to the **Organizational Settings Tab**.

Home > Users > External Identities

External Identities | Cross-tenant access settings

Privat - Microsoft Entra ID for workforce

Search Got feedback?

Overview

Cross-tenant access settings

All identity providers

External collaboration settings

Diagnose and solve problems

Self-service sign up

Custom user attributes

All API connectors

Custom authentication extensions (Preview)

User flows

Subscriptions

Linked subscriptions

Lifecycle management

Terms of use

Access reviews

Troubleshooting + Support

Organizational settings Default settings Microsoft cloud settings

+ Add organization Refresh Columns

Use cross-tenant access settings to manage collaboration with external Microsoft Entra tenants. For non-Microsoft Entra tenants, use collaboration settings. [Edit or view collaboration restrictions](#)

Organizational settings are cross-tenant access settings you've configured for specific Microsoft Entra tenants. Any Microsoft Entra tenants not listed here will use the default settings. [Learn more](#)

Search by domain name or tenant ID

0 organizations found

Name	Inbound access	Outbound access	Tenant restrictions
Add your first organization to configure organization-specific settings			

Add organization

Once there, you can click on "Add Organization".

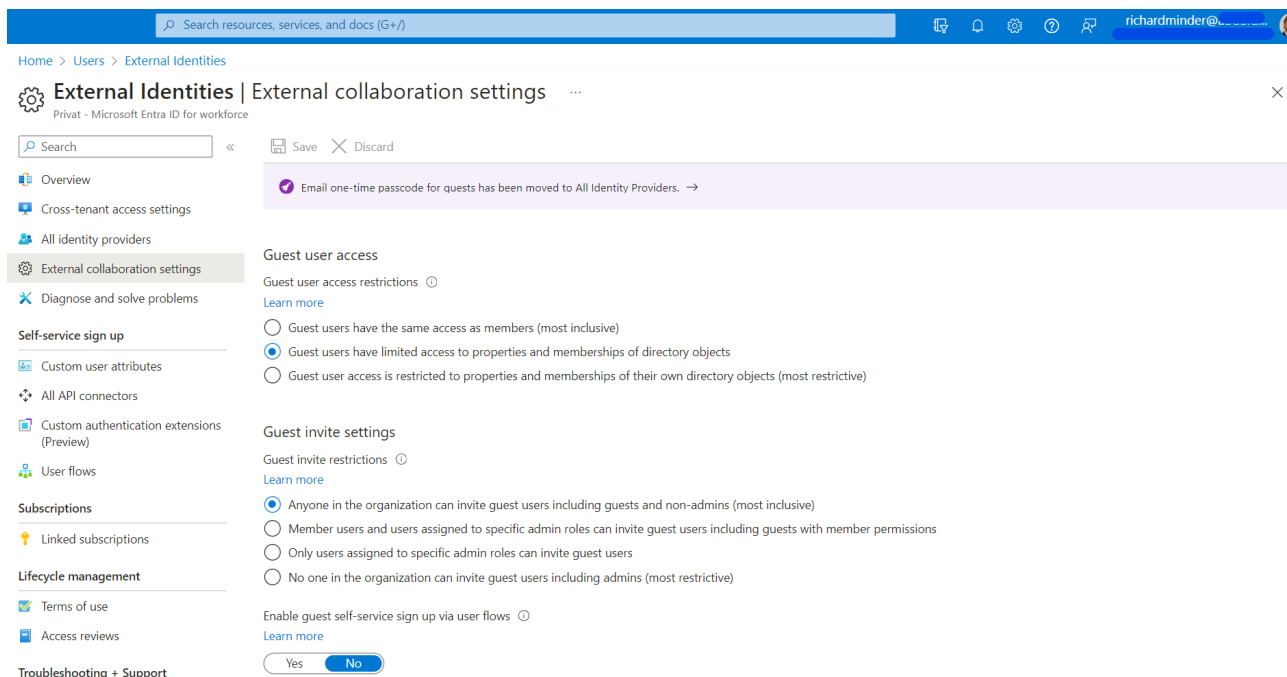
Make sure you have the Tenant-ID or Domain Name of your target organization ready

3 - Reviewing External Collaboration Settings#

Now we ensure, that we know and decide:

- What access do the Guest Users have?
- Who can invite Guests?
- Can Guests signup via Self-Service Signup?
- Can Guests remove themselves?
- Which Domains can be invited?

On the left menu bar, navigate to **External Collaboration Settings**:

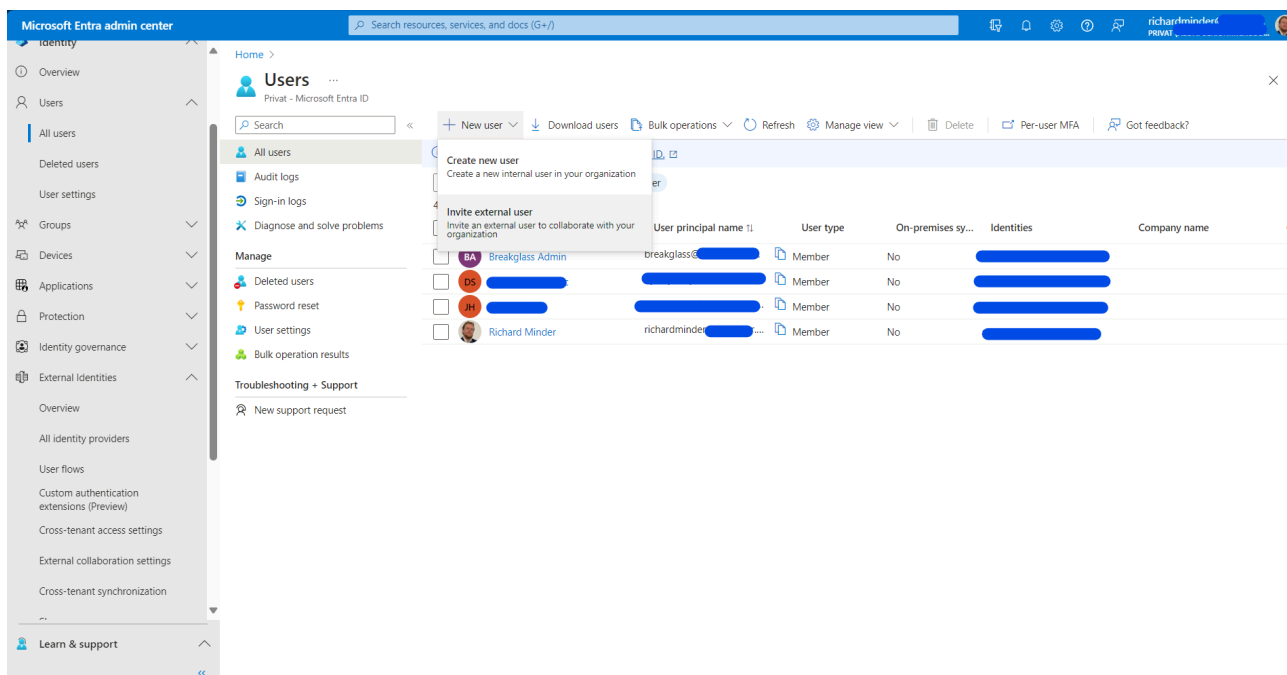


4 - Inviting Users#

Now that we configured the Default Access Settings, the Collaboration Settings and added our first organization, we can start inviting users.

The invitation happens through an invite link which gets sent via email. You can configure a custom invite message for that mail.

Navigate to **Identity > Users > All Users**.



Then, add a new user and select **Invite External User**.

Deep Dive - How does it work?#

Similar to on-premises Active Directory networks, authentication and authorization within and across tenants works using tickets.

These tickets come in the form of a cookie stored in our webbrowser. This cookie contains information about our identity in the form of an Access Token, which is a **JSON Web Token, or JWT**. JWT's are a suitable solution for this job, since they can be signed.

A signed JWT consists of three parts:

- The header, which contains information about the JWT, such as which signing algorithm was used.
- The payload contains information in form of a JSON object.
- The signature is used to verify the integrity of the JWT.

The issuer of the JWT signs it using his private key. Thanks to Public Key Encryption, the integrity of the JWT can be checked using the public key of our tenant.

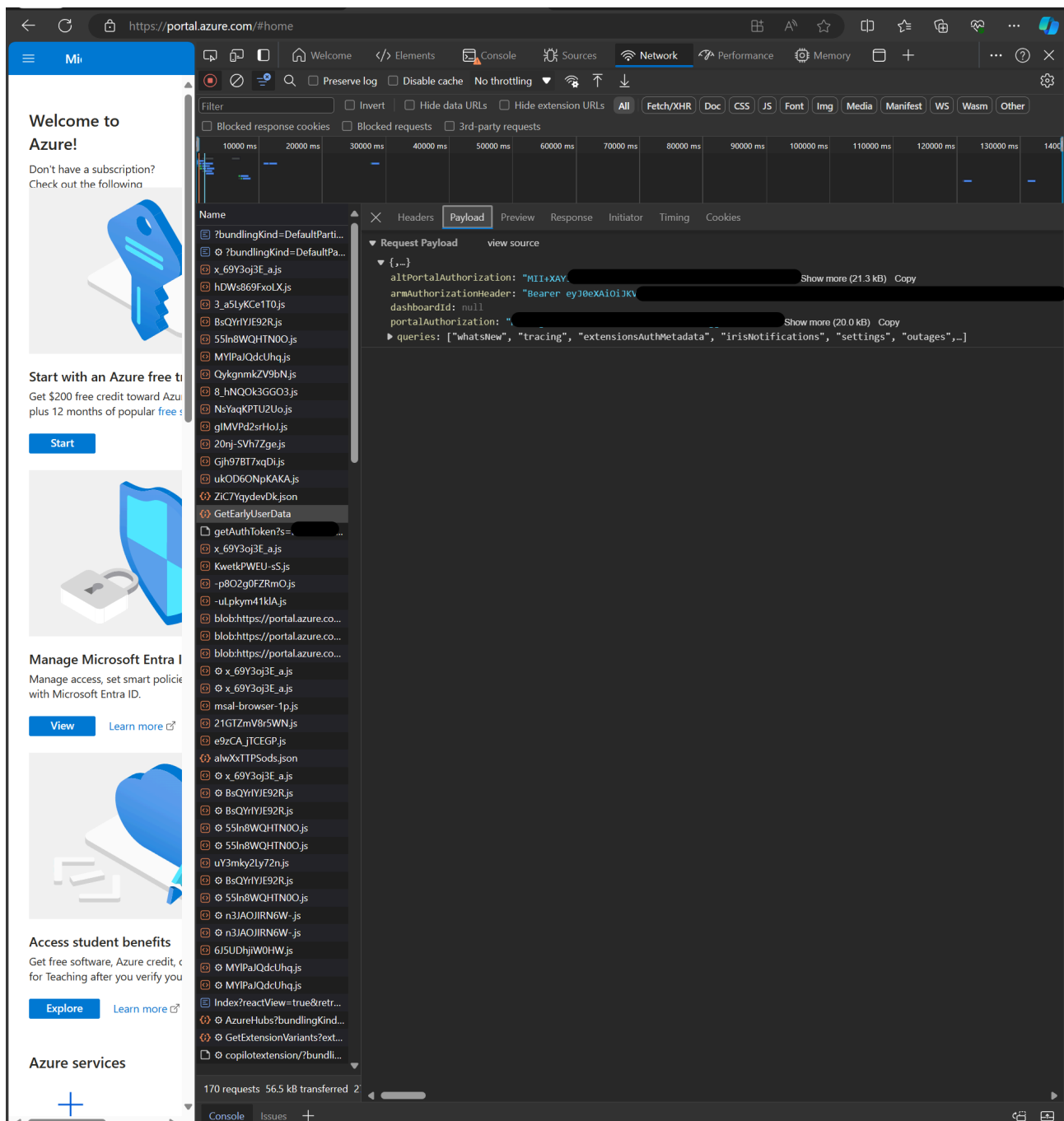
Azure JWT Tickets#

Our home tenant stores information about our session inside the JWT and then signs it. The signing process ensures that the cookie cannot be tampered with using public-key cryptography. Our home tenant simply generates the JWT, hashes it and then uses its private key to encrypt the hash.

The mechanism is similar to an employee walking through an office complex with his ID card. When we want to open a door and access an office room, we have to hold our card against the scanner next to the door. Some doors open, others don't, depending on our access level.

Just like that access card, we use our JWT to move throughout our cloud resources. As soon as we try to access a resource, we submit our JWT using a web request. The resource then checks whether our identity is supposed to have access, and then either grants us access or denies it.

Let's see for ourselves. Open Developer Tools in your browser of choice, go to the Network-Tab and search for the "**GetEarlyUserData**"-Post request. Then head into the **Payload**-Tab:



The **armAuthorizationHeader** is holding our "Ticket". Copy the value and remove the "Bearer "-String in front of the actual JWT, then post it into your JWT-Analyzing tool of choice. I chose jwt.ms, since this JWT was issued by Microsoft and I just feel more comfortable sharing it only with Microsoft:

[illegible]

Decoded Token		Claims
Claim type	Value	Notes
aud	https://management.core.windows.net/	Identifies the intended recipient of the token. In id_tokens, the audience is your app's Application ID, assigned to your app in the Azure portal. Your app should validate this value, and reject the token if the value does not match.
iss	https://sts.windows.net/[REDACTED]b/	Identifies the security token service (STS) that constructs and returns the token, and the Azure AD tenant in which the user was authenticated. If the token was issued by the v2.0 endpoint, the URI will end in /v2.0. The GUID that indicates that the user is a consumer user from a Microsoft account is 9188040d-6c67-4c5b-b112-36a304b66dad. Your app should use the GUID portion of the claim to restrict the set of tenants that can sign in to the app, if applicable.
iat	Sat Apr 06 2024 13:54:13 GMT+0200 (Central European Summer Time)	"Issued At" indicates when the authentication for this token occurred.
nbf	Sat Apr 06 2024 13:54:13 GMT+0200 (Central European Summer Time)	The "nbf" (not before) claim identifies the time before which the JWT MUST NOT be accepted for processing.
exp	Sat Apr 06 2024 15:04:05 GMT+0200 (Central European Summer Time)	The "exp" (expiration time) claim identifies the expiration time on or after which the JWT MUST NOT be accepted for processing. It's important to note that a resource may reject the token before this time as well - if for example a change in authentication is required or a token revocation has been detected.
acr	1	The "Authentication context class" claim. A value of "0" indicates the end-user authentication did not meet the requirements of ISO/IEC 29115.
aio	AM[REDACTED]	An internal claim used by Azure AD to record data for token reuse. Should be ignored.

The red part of the JWT represents the header, the blue part the payload and the green part the signature.

Claims#

The JWT issued by our tenant stores so-called "Claims". These claims are very important when accessing resources in our tenant, as well as when accessing other tenants.

In the first picture above, you can see:

- The intended recipient
- The ISS, which is the tenant in which my account was authenticated
- The expiration date

When scrolling down, we can find further claims which are important for Cross-Tenant Access:

Group Memberships

groups	6a[REDACTED]11-002e-c 44[REDACTED]3cb -41e[REDACTED]c	Provides object IDs that represent the subject's group memberships. These values are unique (see Object ID) and can be safely used for managing access, such as enforcing authorization to access a resource. The groups included in the groups claim are configured on a per-application basis, through the groupMembershipClaims property of the application manifest. A value of null will exclude all groups, a value of "SecurityGroup" will include only Active Directory Security Group memberships, and a value of "All" will include both Security Groups and Office 365 Distribution Lists. See the hasgroups claim below for details on using the groups claim with the implicit grant. For other flows, if the number of groups the user is in goes over a limit (150 for SAML, 200 for JWT), then an overage claim will be added to the claim sources pointing at the Graph endpoint containing the list of groups for the user.
--------	---	---

Authentication Tokens (Here, Password and MFA)

amr	pwd,mfa	Identifies how the subject of the token was authenticated. Microsoft identities can authenticate in a variety of ways, which may be relevant to your application. The amr claim is an array that can contain multiple items, such as ["mfa", "rsa", "pwd"], for an authentication that used both a password and the Authenticator app. See the amr claim section in Azure Active Directory access tokens documentation for values.
-----	---------	---

IP-Address, Name, Identifier

idtyp	user	
ipaddr	2[REDACTED]3:b 7[REDACTED]1c	The IP address the user authenticated from.
name	Richard Minder	The name claim provides a human-readable value that identifies the subject of the token. The value is not guaranteed to be unique, it is mutable, and it's designed to be used only for display purposes. The profile scope is required in order to receive this claim.
oid	e[REDACTED]88-7 5[REDACTED]69	The immutable identifier for an object in the Microsoft identity system, in this case, a user account. This ID uniquely identifies the user across applications - two different applications signing in the same user will receive the same value in the oid claim. The Microsoft Graph will return this ID as the id property for a given user account. Because the oid allows multiple apps to correlate users, the profile scope is required in order to receive this claim. Note that if a single user exists in multiple tenants, the user will contain a different object ID in each tenant - they are considered different accounts, even though the user logs into each account with the same credentials.

Tenant-ID, Unique Name, UPN

tid	b[REDACTED]8-c c[REDACTED]b	A GUID that represents the Azure AD tenant that the user is from. For work and school accounts, the GUID is the immutable tenant ID of the organization that the user belongs to. For personal accounts, the value is 9188040d-6c67-4c5b-b112-36a304b66dad. The profile scope is required in order to receive this claim.
unique_name	richardminder@[REDACTED].on microsoft.com	Provides a human readable value that identifies the subject of the token. This value is not guaranteed to be unique within a tenant and should be used only for display purposes. Only issued in v1.0 id_tokens.
upn	richardminder@[REDACTED].on microsoft.com	The username of the user. May be a phone number, email address, or unformatted string. Should only be used for display purposes and providing username hints in reauthentication scenarios.

When accessing another tenant, these claims will still be stored in our JWT. As soon as we try to access resources from that tenant, our JWT will be checked.

Thank you for reading my Post, I hope you learned something new and are now able to implement a secure Cross-Tenant Access solution!